Fast PyMC 5.x sampling: nutpie, NumPyro, and BlackJAX on your workstation

For your small horseshoe regression (n=46, ≤10 predictors), nutpie is the clear winner — delivering ~2× speedup over default PyMC NUTS with zero GPU overhead and no multiprocessing headaches. GPU acceleration via JAX is counterproductive at this scale; the crossover where GPU outperforms CPU is around 50,000 observations. Pymc Stan Benchmark The reason PyMC runs only 4 chains at a time when you request 8 is a hard-coded default: `cores` caps at `min(cpu_count(), 4)`. PyMC The fix is trivially `pm.sample(chains=8, cores=8)`, but nutpie sidesteps the issue entirely by running Rust threads via Rayon, bypassing Python's multiprocessing. Below is a complete practical guide for your Ubuntu/RTX 3090/Python 3.11 setup.

---

nutpie: the fastest CPU sampler, and your best bet

Nutpie (v**0.16.5**, Feb 2026) is a Rust-based NUTS sampler by Adrian Seyboldt (PyMC Labs) that uses a novel mass matrix adaptation minimizing Fisher divergence. It ships **pre-compiled binary wheels** — no Rust toolchain required. PyPI GitHub On posteriordb benchmarks, it averages **~2× faster than both Stan and default PyMC NUTS,** Pymc-devs with one user reporting a drop from "multiple hours" to ~10 minutes on a hierarchical model. PyMC Discourse

Installation and basic usage are straightforward:

```bash
# Install (pre-built wheels, no Rust needed)
pip install "nutpie[pymc]"
# Or via conda:
conda install -c conda-forge nutpie pymc numba
```

```python
import pymc as pm

with pm.Model() as model:
    # ... your horseshoe model ...
    idata = pm.sample(
        draws=4000,
        tune=2000,
        chains=8,
        target_accept=0.95,
        nuts_sampler="nutpie",
        random_seed=42,
    )
```

Nutpie's parallelism runs entirely in **Rust via Rayon** (a work-stealing thread pool), completely bypassing Python's `multiprocessing` module. pymc This means no GIL issues, no `fork`/`spawn` context problems, and no mysterious 4-chain cap. When you pass `chains=8`, all 8 chains run simultaneously on your 32 CPU cores with no additional configuration. Nutpie defaults to 6 chains if you don't specify.

For GPU support, nutpie offers a JAX backend Pymc-devs (`nutpie.compile_pymc_model(model, backend="jax")`), GitHub but for your model size the Numba CPU backend (the default) will be faster. An experimental **normalizing flow adaptation** feature (`compiled.with_transform_adapt(num_layers=5)`) may help with the funnel geometry of horseshoe priors, though this is still cutting-edge. Pymc-devs Pymc-devs Python 3.11 is fully supported — nutpie actually requires Python ≥ 3.11. PyPI

Known caveat: PyMC emits `UserWarning: Use of external NUTS sampler is still experimental`. PyMC This warning has persisted for years; the feature is production-grade and widely used at PyMC Labs. PyMC

NumPyro via JAX: powerful but overkill for small models

NumPyro (v**0.20.0**, Jan 2026) integrates with PyMC through `pm.sample(nuts_sampler="numpyro")`. PyPI PyMC transpiles the PyTensor model graph to JAX, then hands the compiled log-probability function to NumPyro's NUTS kernel. Medium On CPU alone, JIT compilation eliminates Python overhead and yields **2–3× ESS/second improvement** over default PyMC. Pymc Stan Benchmark On GPU with large datasets, the PyMC Labs benchmark measured **~11× more effective samples/second** than default PyMC. Pymc Stan Benchmark +2

However, **GPU acceleration is counterproductive for your use case**. With n=46 and 5–10 parameters, GPU kernel launch overhead far exceeds compute savings. The RTX 3090 also has a **64:1 FP32-to-FP64 throughput ratio** — since JAX defaults to float64 for MCMC numerical stability, you'd use roughly 1.5% of the card's theoretical performance. Martiningram The well-documented crossover point is **~50,000 observations** before GPU becomes beneficial. Pymc Stan Benchmark Pymc Stan Benchmark

If you still want NumPyro available as a fallback, there is a critical configuration detail for running 8 chains on a single GPU:

```python
# CORRECT: vectorized chains on single GPU
idata = pm.sample(
    draws=4000, tune=2000, chains=8,
    nuts_sampler="numpyro",
    nuts_sampler_kwargs={"chain_method": "vectorized"},
    target_accept=0.95,
)

# WRONG: "parallel" with 1 GPU silently falls back to sequential
# chain_method="parallel" uses jax.pmap — one chain per device.
# With 1 GPU and 8 chains, NumPyro runs chains ONE AT A TIME.
```

The `chain_method` distinction matters enormously. `"parallel"` maps chains across devices via `jax.pmap` — on a single GPU, this silently degrades to sequential execution. Pyro `"vectorized"` uses `jax.vmap` to batch all chains on one device. pyro +3 But note that `vmap` synchronizes chains at each leapfrog step, so the slowest chain's trajectory length bottlenecks all others. Blackjax-devs The BlackJAX tutorial measured **pmap as 43× faster than vmap** for NUTS with 32 CPU chains. Statistical Modeling Blackjax-devs For CPU-only NumPyro, force multiple virtual devices:

```bash
export XLA_FLAGS="--xla_force_host_platform_device_count=8"
```

JAX CUDA installation for RTX 3090

The JAX team explicitly recommends pip over conda for GPU installs. The pip wheels bundle CUDA and cuDNN — no system CUDA toolkit required. JAX Documentation Your RTX 3090 (Ampere, SM 8.6) needs NVIDIA driver ≥ 525 for CUDA 12: JAX Documentation

```bash
pip install --upgrade "jax[cuda12]"

# Verify GPU detection
python -c "import jax; print(jax.devices()); print(jax.default_backend())"
# Expected: [CudaDevice(id=0)]  gpu
```

Version pinning for stability: PyMC **5.28.2** + NumPyro **0.20.0** + JAX **0.9.2** + Python 3.11 is the tested combination as of March 2026. Older JAX versions may trigger `NotImplementedError: jax.experimental.host_callback has been deprecated` — ensure all packages are current. pymc PyMC Discourse

---

BlackJAX: GPU-capable but less stable

BlackJAX (v**1.3**, late 2025) is a modular JAX-based sampler library. Pymc Stan Benchmark Usage mirrors NumPyro:

```python
idata = pm.sample(
    draws=4000, tune=2000, chains=8,
    nuts_sampler="blackjax",
    nuts_sampler_kwargs={
        "chain_method": "vectorized",
        "postprocessing_backend": "cpu",
    },
)
```

BlackJAX offers the same GPU capabilities and `pmap`/`vmap` chain parallelism as NumPyro. PyPI PyMC In PyMC's own PPCA benchmark, BlackJAX clocked **~13s** versus NumPyro's **~12s** PyMC and nutpie's **~16s** (all on CPU, 4 chains). The performance difference is marginal.

The critical issue with BlackJAX is stability. A reported PyMC Discourse thread documents BlackJAX producing divergences with `TruncatedNormal` likelihoods that PyMC default and nutpie handle cleanly. PyMC Discourse For horseshoe priors with their challenging funnel geometry, this stability concern is significant. BlackJAX is best treated as a third-tier fallback.

Why PyMC caps at 4 parallel chains (and how to fix it)

The `cores` parameter in `pm.sample()` defaults to `min(cpu_count(), 4)` — a deliberate cap. PyMC On your 32-core machine with `chains=8` and no `cores` argument, PyMC prints `"Multiprocess sampling (8 chains in 4 jobs)"` and runs two sequential batches of 4. Duke University The fix for the default sampler is explicit:

```python
# Default PyMC NUTS: must specify cores to override the cap
idata = pm.sample(chains=8, cores=8, draws=4000)
```

The `mp_ctx` parameter (`"fork"`, `"spawn"`, `"forkserver"`) controls the multiprocessing context. On Linux, it defaults to `"fork"`, which is generally fine. A February 2026 Discourse thread reported multiprocessing failures on Linux with Python 3.14, where even `mp_ctx="forkserver"` didn't help — the recommended workaround was switching to nutpie. pymc

Alternative samplers bypass this entirely. Nutpie uses Rayon's Rust thread pool. PyPI pymc NumPyro and BlackJAX use JAX's `pmap`/`vmap`. Medium +2 None of them touch Python's `multiprocessing`, so the `cores` cap is irrelevant. With nutpie, `chains=8` means 8 simultaneous Rust threads — full stop.

Horseshoe priors demand specific sampler tuning

The Finnish (regularized) horseshoe prior creates **funnel geometry** where the global shrinkage τ compresses all local parameters near zero. Readthedocs This is a notorious sampling challenge. Three practices are essential regardless of sampler:

- **Non-centered parameterization** — write $\beta = z \cdot \tau \cdot \tilde{\lambda}$ where $z \sim \text{Normal}(0,1)$, letting the sampler explore the funnel without pathological curvature (PyMC)
- **Regularized horseshoe** (Piironen & Vehtari 2017) — adds a slab component via InverseGamma that caps coefficient magnitudes, dramatically reducing divergences (Taylor & Francis Online)
- **High target acceptance** — set `target_accept=0.95` or higher to force smaller step sizes that navigate the funnel neck (PyMC)

Nutpie's novel mass matrix adaptation (minimizing Fisher divergence rather than sample covariance) (GitHub) and its support for **low-rank plus diagonal mass matrices** may handle funnel geometry better than standard diagonal mass matrices used by default in PyMC and Stan. (Statistical Modeling)

```python
import pymc as pm
import numpy as np

def build_horseshoe_model(X, y, D0_prior=3):
    """Regularized horseshoe (Piironen & Vehtari 2017), non-centered."""
    N, D = X.shape
    with pm.Model() as model:
        sigma = pm.HalfNormal("sigma", sigma=2.0)

        # Global shrinkage — prior scale from expected sparsity
        tau0 = (D0_prior / (D - D0_prior)) * (sigma / np.sqrt(N))
        tau = pm.HalfStudentT("tau", nu=2, sigma=tau0)

        # Local shrinkage
        lam = pm.HalfStudentT("lam", nu=5, sigma=1.0, shape=D)

        # Slab (regularization for large coefficients)
        c2 = pm.InverseGamma("c2", alpha=2, beta=1)
        lam_tilde = pm.math.sqrt(
            c2 * lam**2 / (c2 + tau**2 * lam**2)
        )

        # Non-centered parameterization
        z = pm.Normal("z", 0, 1, shape=D)
        beta = pm.Deterministic("beta", z * tau * lam_tilde)

        beta0 = pm.Normal("beta0", mu=0, sigma=10)
        mu = beta0 + pm.math.dot(X, beta)
        pm.Normal("obs", mu=mu, sigma=sigma, observed=y)
    return model
```

Complete installation commands for your environment

```bash
# === Create environment ===
conda create -c conda-forge -n pymc_env python=3.11 -y
conda activate pymc_env

# === Install PyMC ecosystem via conda-forge ===
conda install -c conda-forge "pymc>=5" arviz matplotlib -y

# === Install nutpie (primary sampler) ===
conda install -c conda-forge nutpie numba -y

# === Install JAX-based samplers ===
conda install -c conda-forge numpyro blackjax -y

# === GPU support: JAX CUDA 12 (pip recommended by JAX team) ===
# This bundles CUDA/cuDNN — no system CUDA toolkit needed
# Requires NVIDIA driver >= 525 (check: nvidia-smi)
pip install --upgrade "jax[cuda12]"

# === Verify everything ===
python << 'EOF'
import pymc as pm; print(f"PyMC {pm.__version__}")
import nutpie;      print("nutpie OK")
import numpyro;     print(f"NumPyro {numpyro.__version__}")
import blackjax;    print("BlackJAX OK")
import jax
print(f"JAX {jax.__version__}, backend: {jax.default_backend()}")
print(f"Devices: {jax.devices()}")
EOF
```

Gotcha on mixing conda + pip for JAX: Install all conda packages first, then pip-install `jax[cuda12]` last. The pip wheels override conda's CPU-only jaxlib. If you encounter `LD_LIBRARY_PATH` conflicts, unset it — pip-bundled CUDA should be self-contained. JAX Documentation

All-pip alternative (simpler dependency management):

```bash
conda create -n pymc_env python=3.11 -y && conda activate pymc_env
pip install "pymc>=5" "nutpie[pymc]" numpyro blackjax "jax[cuda12]" arviz
```

Sampler fallback pattern: production-ready code

python

```python
import pymc as pm
import logging

log = logging.getLogger(__name__)

def sample_with_fallback(
    model, chains=8, draws=4000, tune=2000,
    target_accept=0.95, random_seed=42,
):
    """Try samplers: nutpie → numpyro → default PyMC NUTS."""
    kw = dict(
        chains=chains, draws=draws, tune=tune,
        target_accept=target_accept, random_seed=random_seed,
        return_inferencedata=True,
    )

    # 1. nutpie — fastest CPU, Rust threads, no multiprocessing
    try:
        log.info("Attempting nutpie...")
        with model:
            idata = pm.sample(nuts_sampler="nutpie", **kw)
        log.info("nutpie succeeded")
        return idata, "nutpie"
    except (ImportError, ModuleNotFoundError) as e:
        log.warning(f"nutpie unavailable: {e}")
    except Exception as e:
        log.warning(f"nutpie failed: {e}")

    # 2. numpyro — JAX JIT on CPU still gives 2-3x speedup
    try:
        log.info("Attempting numpyro...")
        with model:
            idata = pm.sample(nuts_sampler="numpyro", **kw)
        log.info("numpyro succeeded")
        return idata, "numpyro"
    except (ImportError, ModuleNotFoundError) as e:
        log.warning(f"numpyro unavailable: {e}")
    except Exception as e:
        log.warning(f"numpyro failed: {e}")

    # 3. Default PyMC NUTS — always available, override cores cap
```

```
    log.info("Falling back to default PyMC NUTS")
    with model:
        idata = pm.sample(nuts_sampler="pymc", cores=chains, **kw)
    return idata, "pymc"
```

The recommended order is **nutpie → numpyro → default PyMC**. BlackJAX is intentionally omitted from the primary fallback chain due to its documented stability issues with certain likelihoods. If you want it as a middle option, slot it between numpyro and default PyMC.

Practical recommendation for your specific workload

For **8 chains × 4,000 draws** of a regularized horseshoe regression with **n=46 and 5-10 predictors**, the expected runtime with nutpie on your 32-core machine is likely **under 2 minutes**. This is a small model where CPU-based sampling dominates. Use `target_accept=0.95` and non-centered parameterization. The RTX 3090 sits idle — and that's the right call. GPU MCMC only pays off above ~50K observations where gradient computation (large matrix operations) dominates over per-step overhead. Pymc Stan Benchmark +2 Your bottleneck is funnel geometry, not compute.